

An Efficient Algorithm for Frequent Closed Itemsets Mining

Lisheng Ma

Department of Computer Science and Technology
Chuzhou University
Chuzhou, Anhui, China
mls20041084@sina.com

Yi Qi

School of Computer
ChongQing University Of Arts And Sciences
Chongqing, China
qy0111@126.com

Abstract

Efficient algorithms for mining frequent itemsets are crucial for mining association rules. Most existing work focuses on mining all frequent itemsets. However, since any subset of a frequent set also is frequent, it is sufficient to mine the set of frequent closed itemsets which determines exactly the complete set of all frequent itemsets and is usually much smaller than the latter. In this paper, we study the performance of the existing approaches for mining frequent closed itemsets. We also develop an algorithm, called FCFIA. In this algorithm, we develop and integrate two techniques in order to improve the efficiency of mining frequent closed itemsets. We also present experimental results which show that our method outperforms the existing methods FPclose.

Keywords: Data mining, Association Rules, Frequent Closed Itemsets.

1. Introduction

Let $I = \{i_1, i_2, \dots, i_n\}$ be a set of items, We call $X \subseteq I$ an itemset, and we call X a k -itemset if the cardinality of itemset X is k . Let database T be a multiset of subsets of I , and let $\text{support}(X)$ be the percentage of itemset Y in T such that $X \subseteq Y$. Informally, the support of an itemset measures how often X occurs in the database. **If $\text{support}(X) \geq \text{minSup}$, we say that X is a frequent itemset, and we denote the set of all frequent itemsets by FI . If X is frequent and none of the frequent superset of X have the same support, we say that X is a frequent closed itemset, and we denote the set of all frequent closed itemsets by CFI .**

There are basically two types of algorithms to mine frequent itemsets, breadth-first algorithms and depth-first algorithms. The breadth-first algorithms, such as Apriori [1, 2] and its variants, apply a bottom-up level-wise search in the itemset lattice. On the other hand, depth-first algorithms such as FP-growth [3] search the lattice bottom-up in depth-first way. From a singleton itemset $\{i\}$, successively larger candidate sets are generated by adding one element at a time.

The drawback of mining all frequent itemsets is that if there is a large frequent itemset with size l , then almost all 2^l candidate subsets of the itemset might be generated. However, The set of frequent closed itemsets uniquely determines the exact frequency of

all itemsets, yet it can be orders of magnitude smaller than the set of all frequent itemsets, it is sufficient to discover only all frequent closed itemsets.

Close [4] is an Apriori-like algorithm that directly mines frequent closed itemsets. There are two main steps in Close. The first is to use bottom-up search to identify generators, the smallest frequent itemset that determines a closed itemset. All generators are found using a simple modification of Apriori. After finding the frequent sets at level k , Close compares the support of each set with its subsets at the previous level. If the support of an itemset matches the support of any of its subsets, the itemset cannot be a generator and is thus pruned. The second step in Close is to compute the closure of all the generators found in the first step. To compute the closure of an itemset we have to perform an intersection of all transactions where it occurs as a subset. The closures for all generators can be computed in just one database scan, provided all generators fit in memory. Nevertheless computing closures this way is an expensive operation.

Gösta Grahne and Jianfei Zhu [8] introduces FPclose, which an extension of the FP-growth method, for mining CFI only. During the mining process, an FP-tree (Frequent Pattern tree) is used to store the frequency information of the whole dataset. To test if a frequent itemset is closed, another structure, called a Closed Frequent Itemset tree (CFI-tree), is utilised to keep track of all frequent closed itemsets. This structure makes FPclose effectively reduce the search time and the number of subset testing operations.

1.1. Contributions

In this paper we introduce FCFIA, an extension of the FP-growth method, for mining CFI only. During the mining process, an FP-tree (Frequent Pattern tree) is used to store the frequent information of the whole dataset. In this paper, we develop and integrate the following two techniques in order to improve the efficiency of mining frequent closed itemsets: 1 we use a strategy like PEP in the algorithm MAFIA, the strategy not only reduce the search space, but also reduce the size of FP-tree; 2 The CFI-tree in the algorithm FPclose is used, which stores all frequent closed itemsets, but we present a method of projection which can save comparison times of items in closed checking.

Experimental results also show that FCFIA is competitive. For datasets connect, it outperforms algorithms FPclose.

1.2. Overview

The rest of the paper is organized as follows. Section 2 gives a brief introduction of the FP-tree and the FP-growth method of [3], and then introduces the structure for storing CFI. In Section 3 we give some optimization strategies. The FCFIA algorithm is also give in this section. In Section 4 we compare our algorithm to the algorithms FPclose, and we found our algorithm is valid in mining frequent closed itemsets. Conclusions is given in Section 5.

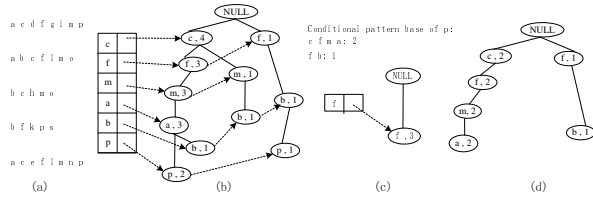


Figure 1. FP-tree ($minSup=3$)

2. Related theory

2.1. FP-tree and FP-growth method

In the aforementioned FP-growth method [3], a novel data structure, the FP-tree (Frequent Pattern tree) is used. The FP-tree is a compact data structure for storing all necessary information about frequent itemsets in a database. Every branch of the FP-tree represents a frequent itemset, and the nodes along the branch are ordered decreasingly by the frequency of the corresponding item, with leaves representing the least frequent items. Each node in the FP-tree has three fields: *item-name*, *count* and *node-link*, when *item-name* registers which item this node represents, *count* registers the number of transactions represented by the portion for the path reaching this node, and *node-link* links to the next node in the FP-tree carrying the same *item-name*, or null if there is none. The FP-tree has a header table associated with it. Single items are stored in the header table in decreasing order of frequency. Each entry in the header table consists of two fields, *item-name* and head of *node-link* (a pointer pointing to the first node in the FP-tree carrying the *item-name*).

Compared with Apriori and its variants which need several database scans, the FP-growth method only needs two database scans when mining all frequent itemsets. In the first scan, all frequent items are found. The second scan constructs the first FP-tree which contains all frequency information of the original dataset. Mining the database then becomes mining the FP-tree. Figure 1 (a) shows a database example. After the first scan, all frequent items are inserted into the header table of an initial FP-tree. Figure 1 (b) shows the first FP-tree constructed from the second scan. More details about the construction of FP-tree and FP-growth method can be found in [3].

When add an item i to the existing itemset $head$, we denote the itemset $head \cup i$ by Z , the path from the parent node of this

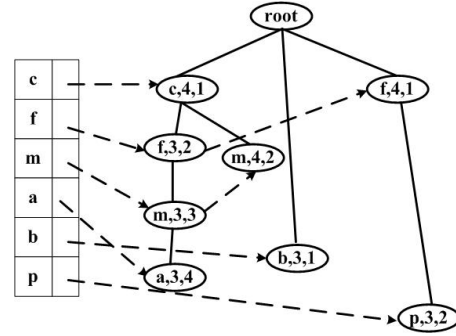


Figure 2. CFI-tree ($minSup=3$)

node (node's *item-name* is i) to the root node in the *head*'s FP-tree is called Z 's prefix path. Figure 1(d) shows the prefix paths for item $\{p\}$.

2.2. CFI-tree

In algorithm FPclose introduces the Closed Frequent Itemset tree (CFI-tree) as the special data structure to store CFI. The CFI-tree resembles an FP-tree. It has a root labelled with "root". Children of the root are item prefix subtrees. Each node in the subtree has four fields: *item-name*, *count*, *node-level* and *node-link*. All nodes with same *item-name* are linked together. The *node-link* points to the next node with same *item-name*. A header table is constructed for items in the CFI-tree, the item order in the table is same as the item order in the first FP-tree constructed from the first scan of the database. Each entry in the header table consists of two fields: *item-name* and head of a *node-link*. The *node-link* points to the first node with the same *item-name* in the CFI-tree. Here, level is still used for subset testing. The count field is needed because when comparing Y with a set Z in the tree, we are trying to verify that it is not the case that $Y \subset Z$ and Y and Z have the same count. In FPclose, a newly discovered frequent itemset is inserted into the CFI-tree, unless it is a subset of an itemset and they have the same count already in the tree. Figure 2 shows the CFI-tree from the database in figure 1(a) when $minSup$ equal 3. In Figure 2, a node x,c,l means that the node is for item x , that its count is c and that its level is l . Due to the lack of space, we omit the algorithm for constructing CFI-tree here.

3. Mining CFI by FCFIA

3.1. Pruning Strategy

Let *head* be an itemset. One method of optimization involves comparing the support of set of items *head* and $head \cup i$ ($i \in head$'s conditional pattern base). If $support(head)$ and $support(head \cup i)$ are equivalence, then any transaction containing *head* also contains i . This guarantees that any frequent itemset Z containing *head* but not i has the frequent superset $Z \cup i$, and they have the same count. According the define of the frequent closed itemset, it is not necessary to count itemsets containing *head* and not i . Therefore, equivalence là cùng xuất hiện trên cùng một Transactions như nhau.

we can move item i to $head$. At the same time, We can also remove item i from the $head$'s conditional pattern base which contain item i .

Another method of optimization involves comparing the support of set of items $head \cup i$ and $Z \cup i$ (Z containing items $head-j$ recently add to $head$). If $support(head \cup i)$ and $support(Z \cup i)$ are equivalence, This guarantees that any frequent itemset $Z \cup i$ has the frequent superset $head \cup i$, they have the same count. According the define of the frequent closed itemset, we can stop the branch search which containing the itemsets $Z \cup i$. (see Pseudocode: *Pruning_Strategy*)

```
Pseudocode: Pruning_Strategy(Current itemset:  $head$ ) {
  for each item  $i$  in  $head$ 's conditional pattern base
  {  $NewItems = head \cup i$ 
    if ( $support(NewItems) == support(head)$ )
      { Move  $i$  to  $head$ 
        Remove  $i$  from  $head$ 's conditional pattern base }
     $NewItems = head \cup i$ 
    if ( $support(NewItems) == support(Z \cup i)$ )
      Stop the branch search that containing the itemset  $Z \cup i$ 
  }
}
```

3.2. Projection Strategy

In the algorithm FPclose, a new structure is used to store frequent closed itemsets, called CFI-tree, which can save comparison times of items in closed checking. But we find it also have some redundancy comparisons in subset checking using CFI-tree, we developed a method of projection can save more comparison times in closed checking.

Let $CFIP[h]$ be a set of the node of CFI-tree. We denote all sets of $CFIP[h]$ ($h \leq |head|$) by $CFIPs$. The set $CFIP[1]$ is initialized as all these nodes in the CFI-tree which contain the first item in $head$. Sort $head$ in support-ascending order.

When a new frequent closed itemset insert into the CFI-tree, for the path which contains the new frequent closed itemset in the CFI-tree, for every node from the leaf node to the root node in this path, if node's $item-name$ is i , i is the h th element in $head$ and the node is not an element in the set $CFIP[h]$, add the node to the set $CFIP[h]$. If node's $item-name$ is i , i is the h th element in $head$ and the node is an element in the set $CFIP[h]$, then stop updating the set $CFIPs$. (see Procedure 1)

When add a new item i to $head$, for every node P in the set $CFIP[h]$, (i is the $h+1$ th item in $head$), if P 's prefix path contains the item i and node Q ($Q.item-name$ is i) in the P 's prefix path is not in the set $CFIP[h+1]$, add Q to the set $CFIP[h+1]$. (see Procedure 2)

Procedure 1 *Update_CFIPs* (leaf, $CFIPs$)

```
Input: Leaf node of CFI-tree; Projection set  $CFIPs$ 
Output: After updating projection set  $CFIPs$ 
(1)  $Q = leaf$ ;  $h = 1$ ;
(2) while ( $Q$  is not root AND  $h \leq |CFIPs|$ ) {
(3) if ( $Q.item-name$  is the  $h$ th item in  $head$  and  $Q$  is not in  $CFIP[h]$ ) {
(4) Add  $Q$  to  $CFIP[h]$ ;  $h++$ ;
(5)  $Q = Q.parent$ 
(6) else if ( $Q.item-name$  is the  $h$ th item in  $head$  and  $Q$  is in  $CFIP[h]$ )
(7) Return  $CFIPs$ 
(8) Return  $CFIPs$ 
```

Procedure 2 *CFIPs_Projection*(x , $CFIP[h]$)

```
Input: A new item  $x$  ( $x$  is the  $h+1$ th item in  $head$ );
Projection set  $CFIP[h]$ 
Output: New projection set  $CFIP[h+1]$ 
(1) if ( $h+1 < |head|$ ) {
```

```
(2) for ( $j = |head|$ ;  $j > h+1$ ;  $j--$ )
(3)  $CFIP[j] = CFIP[j-1]$ 
(4)  $CFIP[h+1]$  is null
(5) for each node  $P$  in  $CFIP[h]$  {
(6) if ( $P$ 's prefix path contains the item  $x$  and node  $Q$  ( $Q.item-name$  is  $x$ ) in the  $P$ 's prefix path is not in the set  $CFIP[h+1]$ )
(7) Add  $Q$  to  $CFIP[h+1]$ 
(8) Return  $CFIP[h+1]$ 
```

3.3. FCFIA: Mining CFI

We extend the FP-growth method and get algorithm FCFIA described in Procedure 3. Like FP-growth, algorithm FCFIA is also recursive. In the initial call, an FP-tree is constructed from the first scan of the database. A linked list $head$ contains the items that form the conditional pattern base of the current call. If there is only one single path in the FP-tree, every frequent itemset X generated from this single path together with $head$ is a frequent closed itemset, then we check if itemset $head \cup X$ is a frequent closed itemset. In line 5, if itemset $head \cup X$ is a frequent closed itemset, we insert $head \cup X$ into the CFI-tree and update the set $CFIPs$ using the path of containing the itemset $head \cup X$ in the CFI-tree. If the FP-tree is not a single path tree, then for each item in the header-table, the item is appended to $head$, line 9 calls function $CFIPs.Projection$, and line 10 calls function $closed_checking$ to check if itemset $head$ is a frequent closed itemsets. if itemset $head$ is a frequent closed itemset, we insert $head$ into the CFI-tree and update the set $CFIPs$ using the path of containing the itemset $head$ in the CFI-tree. In line 13 Construct the $head$'s FP-tree, then call function $Pruning_strategy(head)$, FCFIA will be called recursively. Function $closed_checking$ will be explained shortly. Sort $head$ in support-ascending order in the Procedure 3.

Procedure 3 *FCFIA*(T)

```
Input:  $T$ : FP-tree
Global:
CFI-tree: CFI-tree
 $head$ : a linked list of items
 $CFIPs$ : projection set
Output: The CFI-tree which contains CFI
Method:
(1) if  $T$  only contains a single path  $P$  {
(2) Generate all frequent itemset from  $T$ 
(3) for each  $X$  in frequent itemset
(4) if not  $closed\_checking(head \cup X)$  {
(5) Insert  $X$  into CFI-tree
(6) Update  $MFIPs$ 
}
(7) else for each  $i$  in Header-table of  $T$  {
(8) Append  $i$  to  $head$ 
(9)  $CFIPs.Projection(i, CFIP[h])$ 
(10) if not  $closed\_checking(head)$ 
(11) { Insert  $head$  into CFI-tree
(12) Update  $MFIPs$ 
(13) Construct the  $head$ 's FP-tree  $T_{head}$ 
(14)  $Pruning\_strategy(head)$ 
(15)  $FCFIA(T_{head})$ 
(16) Remove  $i$  from  $head$  }
```

3.4. Closed Checking

In FPclose, the implementation of function $closed_checking$ is following. By using the header-table in the CFI-tree, a set S is not necessarily compared with CFI in the CFI-tree. First, S is sorted according to the order of items in header table of CFI-tree. Suppose the sorted S is $\{i_1, i_2, \dots, i_m\}$. From the header table, we

find the node list for i_m . For each node in the list, we check if its count is equal to or greater than S . If it is, we then test if S is a subset of the ancestors of that node.

However, in the new algorithm, we can do better by using the method of projection. First, *head* is sorted according to the order of items in header table of CFI-tree. Suppose the sorted *head* is $\{i_1, i_2, \dots, i_m\}$. For each node in the set $MFIP[h]$ (The h th item is recently added to *head*), we check if its count is equal to the support of *head*. If it is, we then test if *head* is a subset of the ancestors of that node. If $\{i_1, i_2, \dots, i_m\}$ is not a subset of the ancestors of every node in the set $MFIP[h]$ and they have same count, the function *closed_checking* returns false and shows the itemset *head* is a frequent closed itemset. If $\{i_1, i_2, \dots, i_m\}$ is a subset of the ancestors of a node in the set $MFIP[h]$ and they have same count, function *subset_checking* returns true and shows the itemset *head* is not a frequent closed itemset.

4. Experimental Results

The experiments were conducted on a 1.62Ghz AMD Athlon(tm) 64 X2 with 1GB of memory running Microsoft Windows XP Professional. All code was compiled using Microsoft Visual C++ 6.0.

We tested two algorithms FPClosed and FCFIA on dataset: connect. In general, at the higher supports, the pattern length varies between 5-12 items, while at lower supports the patterns contain over 20 items.

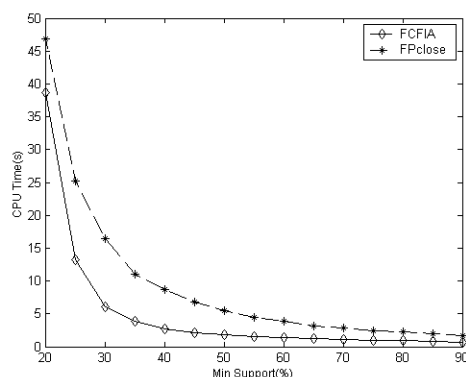


Figure 3. dataset connect

Figure 3 illustrate the running time results of comparing FCFIA to FPClose. We can see our method outperforms the method FPClose on the datasets connect. The x-axis is the user-specified minimum support, while the y-axis is the algorithms running time.

Figure 4 illustrate the percentage results tested on dataset connect. The percentage defined as (the sum of all node in the FP-tree constructed with FP-tree pruning strategy in the algorithm) / (the sum of all node in the FP-tree constructed without FP-tree pruning strategy in the algorithm)

We can see our method can efficiently reduce the size of FP-tree (The x-axis is the user-specified minimum support, while the y-axis is percentage)

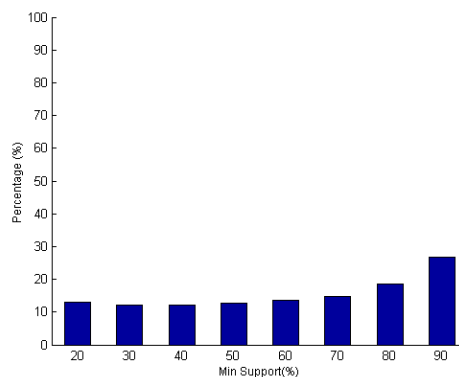


Figure 4. percentage

5. Conclusions

We presented FCFIA, an algorithm for finding frequent closed itemsets. Our experimental results demonstrate that FCFIA consistently outperforms FPClose. In the algorithm FCFIA, the pruning strategy and projection strategy were quite beneficial in reducing the search space, the size of FP-tree and saving comparison times of items.

References

- [1] R. Agrawal, T. Imielinski and A. Swami. Mining association rules between sets of items in large databases. In Proceeding of Special Interest Group on Management of Data, 1993, pages 207-216.
- [2] R. Agrawal and R. Srikant. Fast algorithms for mining association rules. In Proceeding of Int. Conf. Very Large Data Bases. Santiago, Chile, Sept. 1994, pages 487-499.
- [3] J. Han, J. Pei and Y. Yin. Mining frequent patterns without candidate generation. In Proceeding of Special Interest Group on Management of Data, Dallas, TX, May 2000, pages 1-12.
- [4] N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal. Discovering frequent closed itemsets for association rules. In 7th Intl. Conf. on Database Theory, January 1999, pages 398-416.
- [5] J. Pei, J. Han, and R. Mao. Closet: An efficient algorithm for mining frequent closed itemsets. In SIGMOD Intl Workshop on Data Mining and Knowledge Discovery, May 2000, pages 21-30.
- [6] Y. Bastide, R. Taouil, N. Pasquier, G. Stumme, and L. Lakhal. Mining frequent patterns with counting inference. SIGKDD Explorations, 2(2), December 2000, pages 66-75.
- [7] Zaki MJ, Hsiao CJ. CHARM: An efficient algorithm for closed itemset mining. In: Grossman R, et al, eds. Proc. of the 2nd SIAM Intl. Conf. on Data Mining. Arlington: SIAM, 2002. 12-28.
- [8] Grahne G, Zhu J F. Fast Algorithm for Frequent Itemset Mining Using FP-Trees[J]. IEEE Trans.on Knowledge and Data Engineering, 2005, 17(10): 1347-1362.